

Tarea Unidad V — Pruebas funcionales y de carga

Proyecto: todo-api (Node.js/Express) | **Base URL:** http://localhost:8080 (sin prefijo /api) | **Fecha:** 2026-09-08 | **JMeter:** 5.6.3

1. Contrato real de la API

Caso	Método	Ruta	Esperado (esta API)
F01	POST	/tasks	201 + JSON con id y title
F02	GET	/tasks/{id}	200 + mismo id
F03	PUT	/tasks/{id}	200 + completed=true
F04	GET	/tasks	200 + lista JSON
F05	DELETE	/tasks/{id}	204 (sin cuerpo)
F06	GET	/tasks/999999	404 + error controlado
F07	POST	/tasks (sin title)	400 + title obligatorio

La consigna muestra ejemplos 200/201 y /api. Se adaptó al código real: no hay /api, POST crea con 201, DELETE es 204, no existe PATCH.

2. Estructura del Test Plan

```
todo-api funcional
├─ HTTP Request Defaults    host=localhost port=8080 protocol=http
├─ HTTP Header Manager      Content-Type: application/json
├─ CSV Data Set              tareas.csv → ${titulo}, ${descripcion}, ${completada}
├─ TG Funcional CRUD (1 hilo, 1 loop)
│   ├── F01 POST Crear      → JSON Extractor $.id → ${taskId}
│   ├── F02 GET /tasks/${taskId}
│   ├── F03 PUT /tasks/${taskId}
│   ├── F04 GET /tasks
│   ├── F05 DELETE /tasks/${taskId}
│   ├── F06 GET /tasks/999999
│   ├── F07 POST payload inválido
│   ├── View Results Tree
│   └─ Summary Report
└─

todo-api carga
├─ mismos Defaults + Header + CSV
├─ TG Carga (threads/rampup/duration por -J)
│   ├── C01 POST Crear + extraer id
│   ├── C02 GET por id
│   └─ C03 PUT completed=true
```

- └─ C04 GET Listar
- └─ Summary Report
- └─ Aggregate Report

Archivos: jmeter/todo-api-funcional.jmx, jmeter/todo-api-carga.jmx, jmeter/tareas.csv.

3. Prueba funcional (1 usuario, 1 iteración)

7/7 samples OK. Error 0.00%.

Label	Code	Success	Elapsed (ms)
Fo1 POST Crear tarea	201	true	10
Fo2 GET Tarea creada	200	true	0
Fo3 PUT Actualizar tarea	200	true	0
Fo4 GET Listar tareas	200	true	0
Fo5 DELETE Eliminar tarea	204	true	1
Fo6 GET ID inexistente	404	true	0
Fo7 POST payload invalido	400	true	0

Aserciones: código HTTP, existencia de \$.id, title del CSV, completed=true tras PUT, mensaje de error en Fo6/Fo7. El ID se reutiliza con JSON Extractor.

4. Escenarios de carga (justificación)

Se usaron los valores de referencia de la consigna (equipo local, API en memoria, no se buscó saturar). Entre escenarios se reinició el proceso para no mezclar el crecimiento del store con el efecto de la concurrencia.

Escenario	Usuarios	Ramp-up	Duración
Ligera	5	10 s	90 s
Media	20	20 s	120 s
Alta	50	30 s	120 s

En carga no se hace DELETE: evite 404 por carrera entre hilos. Criterio funcional en carga: HTTP 201/200 + JSON con id + completed=true.

5. Métricas (JTL reales, 2026-09-08)

Tiempos en milisegundos. Throughput = samples / ventana del JTL.

Escenario	Samples	Avg	Mediana	p90	p95	Min	Max	Error %	Throughput
Ligera (5)	80756	5.30	4	13	16	0	60	0.000	897.8/s
Media (20)	93380	23.65	19	53	67	0	420	0.000	778.6/s
Alta (50)	93498	56.31	52	120	141	0	676	0.000	779.6/s

Endpoint más lento por escenario (avg)

Escenario	Co1 POST	Co2 GET id	Co3 PUT	Co4 GET Listar
Ligera avg / p95	6.1 / 16	4.3 / 14	3.6 / 13	7.2 / 17
Media avg / p95	24.7 / 69	21.8 / 64	20.7 / 60	27.5 / 70
Alta avg / p95	56.3 / 145	54.9 / 138	54.0 / 136	60.1 / 146

Criterio académico: error < 1% y p95 < 1000 ms. Se cumple en los tres escenarios (p95 máximo 141 ms, errores 0%).

6. Análisis (solo lo que muestran las métricas)

- 1. **Mayor tiempo:** Co4 GET Listar en los tres escenarios. Es coherente con la implementación: serializa *todas* las tareas del Map en cada GET /tasks. Durante cada corrida el store crece (miles de POST), y el avg de Listar sube dentro del mismo escenario (p. ej. ligera: throughput de 1714/s al inicio a 474/s al final).
- 2. **Degradación por concurrencia:** avg total 5.3 → 23.7 → 56.3 ms al pasar de 5 a 20 a 50 hilos. El p95 16 → 67 → 141 ms. No hay umbral de “rotura”; hay aumento gradual. Entre 5 y 20 se nota más el salto relativo.
- 3. **Errores:** 0% en los tres niveles. El porcentaje *no* aumenta con la carga en este experimento.
- 4. **Funcional vs carga:** funcional valida el contrato (códigos y cuerpo) con 1 usuario. Carga mide tiempos/throughput con muchos hilos; las aserciones evitan contar 200 “vacíos” como éxito.
- 5. No se afirma cuello de botella de CPU/red: no se midió. Lo observado es latencia que crece con hilos y con el tamaño de la lista.

7. Uso de IA (prompts y criterio humano)

#	Prompt (adaptado)	Qué se aceptó	Qué se cambió o descartó
1	Casos positivos/negativos/borde para JMeter	F01–F07 y aserciones de código + JSON	No usar /api. POST=201 no 200. DELETE=204. PUT, no PATCH.

2	10 filas CSV titulo,descripcion,completed	Archivo tareas.csv con 10 filas	Se alineó al contrato: completed no se envía en POST (la API lo ignora al crear).
3	Extraer \$.id y reusar en GET/PUT/DELETE	JSON Extractor → \${taskId}	—
4	Escenario académico de carga	5/20/50, ramp 10/20/30, 90–120 s	Sin DELETE en carga (evita 404 por carrera). Reinicio del proceso entre escenarios.
5	Interpretar métricas sin inventar causas	Comparar avg/p95/error/throughput	No se atribuyó el max 676 ms a “GC” ni a red: no hay evidencia. Solo se afirma que Listar es el más lento y que la lista crece.

8. Preguntas de la consigna

11. Endpoint más lento: GET /tasks (Co4). Relevante porque el listado es $O(n)$ sobre el store y se llama en cada iteración de carga.

12. Concurrencia y degradación: ya entre 5 y 20 usuarios el avg pasa de ~5 ms a ~24 ms. A 50 usuarios ~56 ms. Sigue < 1 s.

13. ¿Errores con la carga? No. 0% en ligera, media y alta.

14. Funcional vs carga: funcional = ¿cumple el contrato? Carga = ¿cómo se comporta el tiempo/throughput al subir hilos?

15. IA vs humano: la IA propuso /api y códigos genéricos. El humano los contrastó con app.js.

16. Segunda iteración: paginar GET /tasks; proteger nextId ante concurrencia (hoy ++ no es atómico, aunque en esta corrida no se vieron errores); guardar JTL fuera del repo (son 9–11 MB).

9. Cómo reproducir

```
npm start
# otra terminal:
cd jmeter
jmeter -n -t todo-api-funcional.jmx -l ../results/funcional/funcional.jtl
jmeter -n -t todo-api-carga.jmx -l ../results/carga-ligera/carga.jtl -Jthreads=5 -Jrampup=10 -Jdurati
```

Abrir los .jmx en JMeter GUI para ver el árbol, View Results Tree y Aggregate Report.